

# ASESII 24A02 Project-Based Quiz 1

Ridge, Lasso, and Regularization for Neural Features

Group 04

Nguyen Thi Yen Nhi (24125071), Nguyen Khang Thinh (24125104)

Nguyen Van Tinh (24125106), Nguyen Le Quoc Huy (24125100)

2026-07-16

## Contents

|                                                                   |           |
|-------------------------------------------------------------------|-----------|
| <b>Authorship and contribution statement</b>                      | <b>2</b>  |
| <b>Abstract</b>                                                   | <b>2</b>  |
| <b>Reproducible setup</b>                                         | <b>2</b>  |
| Shared helper functions . . . . .                                 | 3         |
| <b>1 Problem 1. Prediction design and leakage control</b>         | <b>5</b>  |
| 1.1 1A. Target and predictors . . . . .                           | 5         |
| 1.2 1B. Split and train-only preprocessing . . . . .              | 8         |
| 1.3 1C. Training-data exploration . . . . .                       | 9         |
| <b>2 Problem 2. OLS, ridge, and lasso</b>                         | <b>9</b>  |
| 2.1 2A. OLS baseline . . . . .                                    | 9         |
| 2.2 2B. Ridge . . . . .                                           | 13        |
| 2.3 2C. Lasso . . . . .                                           | 13        |
| 2.4 2D. Training-based comparison and locked core model . . . . . | 13        |
| 2.5 2E. Perturbation or stability check . . . . .                 | 13        |
| <b>3 Problem 3. Mathematical mechanisms</b>                       | <b>14</b> |
| 3.1 3A. Ridge and conditioning . . . . .                          | 14        |
| 3.2 3B. Lasso optimality and soft thresholding . . . . .          | 14        |
| 3.3 3C. Connect algebra to evidence . . . . .                     | 14        |
| <b>4 Problem 4. Elastic net and a neural-feature bridge</b>       | <b>14</b> |
| 4.1 4A. Research question and source map . . . . .                | 14        |
| 4.2 4B. Elastic net on original predictors . . . . .              | 14        |
| 4.3 4C. Fixed ReLU features . . . . .                             | 14        |
| 4.4 4D. Locked declaration and final holdout evaluation . . . . . | 15        |
| 4.5 4E. Deep-learning connection and limitations . . . . .        | 15        |
| <b>5 Problem 5. Report quality and reproducibility</b>            | <b>15</b> |

|     |                                              |    |
|-----|----------------------------------------------|----|
| 5.1 | 5A. Reproduction record . . . . .            | 15 |
| 5.2 | 5B. Recommendation and limitations . . . . . | 15 |
| 5.3 | 5C. AI verification record . . . . .         | 15 |

|                                          |           |
|------------------------------------------|-----------|
| <b>Preflight and session information</b> | <b>15</b> |
|------------------------------------------|-----------|

## Authorship and contribution statement

We confirm that every group member can explain the submitted analysis. AI use is reported in Group04\_AI\_Log.csv, and the group checked every AI-assisted item used in this report.

| Student            | ID       | Main contributions | Verification performed |
|--------------------|----------|--------------------|------------------------|
| Nguyen Thi Yen Nhi | 24125071 | [Work]             | [Check]                |
| Nguyen Khang Thinh | 24125104 | [Work]             | [Check]                |
| Nguyen Van Tinh    | 24125106 | [Work]             | [Check]                |
| Nguyen Le Quoc Huy | 24125100 | [Work]             | [Check]                |

## Abstract

*Maximum 150 words: prediction problem, methods, main evidence, and recommendation.*

## Reproducible setup

```
required_packages <- c("tidyverse", "glmnet", "broom", "knitr")
missing_packages <- required_packages[
  !vapply(required_packages, requireNamespace, logical(1), quietly = TRUE)
]
if (length(missing_packages)) {
  stop("Install the required packages before rendering: ",
       paste(missing_packages, collapse = ", "))
}

library(tidyverse)
library(glmnet)
library(broom)
library(knitr)

# `params` exists only while knitting; the fallback
# keeps interactive chunk runs working.
has_params <- exists("params") && !is.null(params$group_number)
group_number <- if (has_params) as.integer(params$group_number) else 4L
if (length(group_number) != 1L || is.na(group_number) || group_number < 1L) {
  stop("Set params$group_number to your assigned positive group number.")
}
```

```

seeds <- list(
  split = 240200L + group_number,
  cv = 240300L + group_number,
  features = 240400L + group_number
)

```

## Shared helper functions

```

find_exam_data <- function(filename) {
  input <- knitr::current_input(dir = TRUE)
  input_dir <- if (length(input) && nzchar(input)) dirname(input) else getwd()
  candidates <- c(
    file.path(input_dir, "data", filename),
    file.path(input_dir, "..", "data", filename),
    file.path(getwd(), "data", filename)
  )
  existing <- candidates[file.exists(candidates)]
  if (!length(existing)) stop("Cannot locate ", filename, " by a relative path.")
  hits <- unique(normalizePath(existing, winslash = "/", mustWork = TRUE))
  if (length(hits) > 1L && length(unique(unname(tools::md5sum(hits)))) > 1L) {
    stop("Multiple nonidentical copies of ", filename, " were found.")
  }
  hits[[1L]]
}

split_rows <- function(n, train_prop = 0.80, seed) {
  stopifnot(n >= 10L, train_prop > 0, train_prop < 1)
  set.seed(seed)
  train <- sort(sample.int(n, floor(train_prop * n), replace = FALSE))
  list(train = train, test = setdiff(seq_len(n), train))
}

fit_scaler <- function(x, tol = 1e-12) {
  x <- as.matrix(x)
  center <- colMeans(x)
  centered <- sweep(x, 2L, center, "-")
  scale <- sqrt(colMeans(centered^2))
  keep <- is.finite(scale) & scale > tol
  if (!any(keep)) stop("No nonconstant training predictor remains.")
  list(
    center = center[keep],
    scale = scale[keep],
    keep = keep,
    dropped = colnames(x)[!keep]
  )
}

```

```

apply_scaler <- function(x, prep) {
  x <- as.matrix(x)[, prep$keep, drop = FALSE]
  x <- sweep(x, 2L, prep$center, "-")
  sweep(x, 2L, prep$scale, "/")
}

make_foldid <- function(n, k = 5L, seed) {
  if (k < 3L || k > n) stop("Require 3 <= k <= number of training rows.")
  set.seed(seed)
  sample(rep(seq_len(k), length.out = n))
}

fit_cv_glmnet <- function(x, y, alpha, foldid) {
  glmnet::cv.glmnet(
    x = x,
    y = y,
    family = "gaussian",
    alpha = alpha,
    foldid = foldid,
    standardize = FALSE,
    intercept = TRUE,
    type.measure = "mse"
  )
}

score_regression <- function(y, prediction) {
  y <- as.numeric(y)
  prediction <- as.numeric(prediction)
  if (length(y) != length(prediction) || any(!is.finite(c(y, prediction)))) {
    stop("Metrics require equal-length finite vectors.")
  }
  c(RMSE = sqrt(mean((y - prediction)^2)),
    MAE = mean(abs(y - prediction)))
}

count_nonzero <- function(fit, s = "lambda.min", tol = 1e-8) {
  b <- as.matrix(stats::coef(fit, s = s))
  slopes <- b[rownames(b) != "(Intercept)", 1L]
  sum(abs(slopes) > tol)
}

safe_condition_numbers <- function(x, lambda, tol = 1e-10) {
  eigenvalues <- eigen(
    crossprod(x) / nrow(x),
    symmetric = TRUE,
    only.values = TRUE
  )$values
  largest <- max(eigenvalues)
}

```

```

smallest <- max(min(eigenvalues), 0)
cutoff <- tol * max(1, largest)
c(
  gram = if (smallest <= cutoff) Inf else largest / smallest,
  ridge = (largest + lambda) / (smallest + lambda)
)
}

```

# 1 Problem 1. Prediction design and leakage control

## 1.1 1A. Target and predictors

```

# The brief assigns prostate.csv to even-numbered groups.
stopifnot(group_number %% 2L == 0L)

config <- list(
  file = "prostate.csv",
  response = "lpsa",
  excluded = "lpsa"
)
data_raw <- read.csv(find_exam_data(config$file), check.names = FALSE)
predictors <- setdiff(names(data_raw), config$excluded)
analysis_data <- data_raw[, c(config$response, predictors), drop = FALSE]

if (anyNA(analysis_data)) stop("Declare a training-only missing-value plan.")
if (!all(vapply(analysis_data, is.numeric, logical(1)))) {
  stop("The assigned response and predictors must be numeric.")
}

```

**Assignment.** Group 04 is even-numbered, so the assigned dataset is `prostate.csv` and the assigned response is `lpsa`.

**Dataset overview.** The `prostate.csv` file originates from the study by Stamey et al. (1989), which examined the relationship between prostate-specific antigen (PSA) levels and several clinical and histological measurements in men with clinically localized prostate cancer who underwent radical prostatectomy. The dataset contains 97 observations and 9 variables — one continuous response (`lpsa`) and eight candidate predictors covering tumour characteristics, patient demographics, and biopsy summaries. All variables are numeric and the dataset has no missing values, so no imputation or factor encoding is required. Individual variable definitions are detailed in the overview table below.

**Response.** `lpsa` is the natural logarithm of prostate-specific antigen (ng/mL), a continuous serum marker measured before surgery. It is modelled on the log scale, so every coefficient is read as a change in log PSA per one standard deviation of a standardized predictor.

**Unit of observation and study population.** One row is one male patient with clinically localized prostate cancer who underwent radical prostatectomy in the study of Stamey et al. (1989). The analysis file contains 97 complete patients. Any conclusion therefore generalizes only to comparable

prostatectomy candidates, not to unscreened men in the general population.

**Prediction objective.** Predict pre-operative `lpsa` from routinely available clinical and biopsy measurements, and compare how OLS, ridge, and lasso trade prediction error against coefficient stability and interpretability when several predictors are correlated.

**Candidate predictors.** All 8 remaining variables are retained as candidate predictors. Every one is numeric, so the template’s type guard passes and the design matrix needs no factor encoding. The variables are drawn from the clinical and pathological records described in Stamey et al. (1989) (Table 1, pp. 1077–1078) and are summarized below.

- `lcavol` — log of cancer volume ( $\log \text{cm}^3$ ), estimated by ultrasound before surgery. Larger tumours are expected to produce more PSA.
- `lweight` — log of total prostate weight ( $\log \text{g}$ ), measured at prostatectomy. Reflects overall prostate size, including both cancerous and non-cancerous tissue.
- `age` — patient age at the time of diagnosis (years).
- `lbph` — log of the amount of benign prostatic hyperplasia ( $\log \text{cm}^2$ ). BPH is non-cancerous prostate enlargement and can independently elevate PSA.
- `svi` — seminal vesicle invasion (binary: 1 = present, 0 = absent), indicating whether cancer has spread beyond the prostate into the seminal vesicles. This is a known marker of advanced disease.
- `lcp` — log of capsular penetration ( $\log \text{cm}$ ), measuring the extent to which cancer has invaded through the prostate capsule. Together with `svi`, it captures local disease progression.
- `gleason` — combined Gleason score (integer, ranging from 6 to 9 in this sample), the standard histological grading system for prostate cancer aggressiveness assigned by a pathologist from biopsy tissue.
- `pgg45` — percentage of Gleason grades 4 or 5 in the biopsy specimen (percent, 0–100). Higher values indicate a larger proportion of poorly-differentiated, aggressive tumour tissue.

All eight predictors were recorded before or during the prostatectomy procedure and are defined independently of the serum PSA measurement, so none introduces outcome leakage. The overview table below records the unit, data type, number of distinct values, and missing-value count for each variable.

**Exclusions.** The project brief requires only that `lpsa` itself be removed from the predictor set, and permits a further exclusion only when a statistical reason is documented. We document none, so exactly one variable is excluded: the response. No candidate predictor is computed from `lpsa`: all 8 are measurements taken before surgery (tumour volume, prostate weight, age, benign hyperplasia, seminal vesicle invasion, capsular penetration, and the two Gleason summaries), and none uses PSA information in its definition.

**Leakage rationale.** A variable computed from the response carries the answer inside the design matrix. Its least-squares coefficient would absorb nearly all explained variance, the training and cross-validation errors of every method would collapse toward zero, and the resulting model would fail on any new patient whose PSA is not yet known — which is precisely the deployment situation. The comparison itself would also become meaningless: OLS, ridge, and lasso would all be fitting the same leaked signal, so differences between them would reflect how each penalty shrinks that one dominant column rather than how each method handles genuine multicollinearity among clinical predictors. Because none of the eight candidate predictors in `prostate.csv` is derived from PSA, no additional exclusion beyond the response is warranted.

```

meaning <- c(
  lcavol = "Log cancer volume",
  lweight = "Log prostate weight",
  age = "Age at diagnosis",
  lbph = "Log BPH amount",
  svi = "Seminal vesicle invasion",
  lcp = "Log capsular penetration",
  gleason = "Gleason score",
  pgg45 = "Percent Gleason grade 4/5",
  lpsa = "Log prostate-specific antigen"
)
unit <- c(
  lcavol = "log cm3", lweight = "log g", age = "years",
  lbph = "log cm2", svi = "0/1", lcp = "log cm",
  gleason = "6 to 9", pgg45 = "percent", lpsa = "log ng/mL"
)

# A column absent from the lookups would otherwise print as a silent blank cell.
stopifnot(all(names(data_raw) %in% names(meaning)),
  all(names(data_raw) %in% names(unit)))

data_overview <- tibble(
  Variable = names(data_raw),
  Role = ifelse(names(data_raw) == config$response, "Response", "Predictor"),
  Meaning = unname(meaning[names(data_raw)]),
  Unit = unname(unit[names(data_raw)]),
  Type = vapply(data_raw, function(x) class(x)[1L], character(1)),
  Distinct = vapply(data_raw, dplyr::n_distinct, integer(1), na.rm = TRUE),
  Missing = vapply(data_raw, function(x) sum(is.na(x)), integer(1))
)

kable(
  data_overview,
  caption = paste("Response and candidate predictors, with the integrity checks",
    "run before the split.")
)

```

Table 2: Response and candidate predictors, with the integrity checks run before the split.

| Variable | Role      | Meaning                  | Unit    | Type    | Distinct | Missing |
|----------|-----------|--------------------------|---------|---------|----------|---------|
| lcavol   | Predictor | Log cancer volume        | log cm3 | numeric | 93       | 0       |
| lweight  | Predictor | Log prostate weight      | log g   | numeric | 88       | 0       |
| age      | Predictor | Age at diagnosis         | years   | integer | 31       | 0       |
| lbph     | Predictor | Log BPH amount           | log cm2 | numeric | 42       | 0       |
| svi      | Predictor | Seminal vesicle invasion | 0/1     | integer | 2        | 0       |
| lcp      | Predictor | Log capsular penetration | log cm  | numeric | 30       | 0       |

| Variable | Role      | Meaning                       | Unit         | Type    | Distinct | Missing |
|----------|-----------|-------------------------------|--------------|---------|----------|---------|
| gleason  | Predictor | Gleason score                 | 6 to 9       | integer | 4        | 0       |
| pgg45    | Predictor | Percent Gleason grade 4/5     | percent      | integer | 19       | 0       |
| lpsa     | Response  | Log prostate-specific antigen | log<br>ng/mL | numeric | 85       | 0       |

```
n_duplicate_rows <- sum(duplicated(analysis_data))
```

**Reading the table.** The file holds 97 patients and 9 columns. Three integrity results clear the way for the split. Every column is numeric, so the type guard above passes. No cell is missing (Missing is zero throughout), so no imputation is performed; the missing-value rule is still declared in 1B as a contingency, because a rule invented after seeing the data would be a decision made with holdout information. And 0 rows are duplicated, so no patient is silently counted twice, which would otherwise inflate the apparent sample size and let the same record fall on both sides of the split. (BPH abbreviates benign prostatic hyperplasia, the non-cancerous prostate enlargement recorded by lbph.)

The **Distinct** column carries the one substantive surprise. Two predictors are numeric but not continuous: **svi** takes only 2 values (a binary indicator of seminal vesicle invasion) and **gleason** only 4 (an ordinal biopsy score from 6 to 9). We keep both on their numeric scale, which is the standard **glmnet** convention, but their coefficients must later be read with that discreteness in mind rather than as smooth slopes: a “one standard deviation” change in a binary variable is a convenient fiction, not a clinical intervention.

The **Unit** column exposes the second issue. The predictors live on wildly different scales, from **pgg45** measured in percent (0 to 100) to several variables already on a log scale. Penalized regression shrinks coefficients toward zero, so a predictor measured in large units would be punished merely for its units, not for being uninformative. This is precisely why the design matrix is standardized in 1B, using centering and scaling constants computed on the training rows only.

## 1.2 1B. Split and train-only preprocessing

```
split <- split_rows(nrow(analysis_data), seed = seeds$split)
train_id <- split$train
test_id <- split$test

x_raw <- as.matrix(analysis_data[, predictors, drop = FALSE])
y_raw <- analysis_data[[config$response]]

x_prep <- fit_scaler(x_raw[train_id, , drop = FALSE])
x_train <- apply_scaler(x_raw[train_id, , drop = FALSE], x_prep)
x_test <- apply_scaler(x_raw[test_id, , drop = FALSE], x_prep)
y_train <- y_raw[train_id]
y_test <- y_raw[test_id]

foldid <- make_foldid(nrow(x_train), k = 5L, seed = seeds$cv)
```

*Report seeds, dimensions, scaling, folds, and leakage safeguards.*



### 1.3 1C. Training-data exploration

```
# Use analysis_data[train_id, ] only. Add focused plots and summaries.
```

Interpret multicollinearity, one anomaly, and the question regularization will address.

## 2 Problem 2. OLS, ridge, and lasso

### 2.1 2A. OLS baseline

```
# Convert x_train (matrix) and y_train into a data frame for lm()
train_data <- data.frame(y = y_train, x_train)

# Fit ordinary least squares on the standardized training set
ols_fit <- lm(y ~ ., data = train_data)

# Tidy coefficient table (term, estimate, std.error, statistic, p.value)
ols_coefs <- broom::tidy(ols_fit)
kable(
  ols_coefs,
  digits = 4,
  caption = "OLS coefficient estimates on standardized training data."
)
```

Table 3: OLS coefficient estimates on standardized training data.

| term        | estimate | std.error | statistic | p.value |
|-------------|----------|-----------|-----------|---------|
| (Intercept) | 2.4757   | 0.0826    | 29.9860   | 0.0000  |
| lcavol      | 0.7228   | 0.1220    | 5.9243    | 0.0000  |
| lweight     | 0.2022   | 0.0971    | 2.0834    | 0.0410  |
| age         | -0.1305  | 0.0949    | -1.3751   | 0.1736  |
| lbph        | 0.1336   | 0.0978    | 1.3657    | 0.1765  |
| svi         | 0.2749   | 0.1160    | 2.3699    | 0.0206  |
| lcp         | -0.0413  | 0.1497    | -0.2757   | 0.7836  |
| gleason     | 0.0365   | 0.1380    | 0.2644    | 0.7923  |
| pgg45       | 0.0945   | 0.1510    | 0.6257    | 0.5336  |

**How to read this table.** Each row is one model term. **estimate** is a *standardized* slope: because every predictor was centred and scaled in 1B, it is the change in **lpsa** for a one standard-deviation increase in that predictor with the others held fixed, so the magnitudes are directly comparable down the column and the intercept equals the mean **lpsa** of the training set. The last three columns judge each estimate — **std.error** is its standard error, **statistic** is the *t*-ratio (**estimate** / **std.error**), and **p.value** tests the null that the coefficient is zero, so a small value flags a predictor whose effect stands out from noise. Watch for any slope whose sign opposes the predictor’s raw correlation with **lpsa**: that is an early fingerprint of the collinearity discussed below.

```

# In-sample fit quality
ols_summary <- summary(ols_fit)

# Training error (optimistic: the model has already seen these rows)
ols_train_pred <- predict(ols_fit, newdata = train_data)
ols_train_metrics <- score_regression(y = y_train, prediction = ols_train_pred)

# Five-fold CV using the SAME foldid later passed to ridge and lasso.
# lm() has no built-in CV, so refit on each four-fold training set and
# score the held-out fold.
K <- max(foldid)
cv_mse_folds <- numeric(K)
for (k in seq_len(K)) {
  idx_train_k <- which(foldid != k)
  idx_val_k <- which(foldid == k)
  fold_data <- data.frame(y = y_train[idx_train_k], x_train[idx_train_k, , drop
↪ = FALSE])
  fold_fit <- lm(y ~ ., data = fold_data)
  fold_pred <- predict(fold_fit,
                        newdata = data.frame(x_train[idx_val_k, , drop =
↪ FALSE]))
  cv_mse_folds[k] <- mean((y_train[idx_val_k] - fold_pred)^2)
}
ols_cv_mse <- mean(cv_mse_folds)
ols_cv_rmse <- sqrt(ols_cv_mse)

# Collect the fit and error numbers into one table
ols_fit_summary <- tibble::tibble(
  Quantity = c("R-squared", "Adjusted R-squared",
               "Training RMSE", "Training MAE",
               "5-fold CV RMSE", "5-fold CV MSE"),
  Value = c(ols_summary$r.squared, ols_summary$adj.r.squared,
            ols_train_metrics[["RMSE"]], ols_train_metrics[["MAE"]],
            ols_cv_rmse, ols_cv_mse)
)
kable(
  ols_fit_summary,
  digits = 4,
  caption = "OLS in-sample fit and five-fold cross-validation error on the shared
↪ folds."
)

```

Table 4: OLS in-sample fit and five-fold cross-validation error on the shared folds.

| Quantity  | Value  |
|-----------|--------|
| R-squared | 0.6729 |

| Quantity           | Value  |
|--------------------|--------|
| Adjusted R-squared | 0.6345 |
| Training RMSE      | 0.6808 |
| Training MAE       | 0.5170 |
| 5-fold CV RMSE     | 0.7935 |
| 5-fold CV MSE      | 0.6297 |

**How to read this table.** The first two rows measure in-sample fit: **R-squared** is the fraction of training variance in `lpsa` the model explains, and **Adjusted R-squared** discounts it for the eight predictors so that adding an uninformative one cannot inflate the score. The lower rows are prediction error on the scale of `lpsa`: **RMSE** penalizes large misses (it is the square root of MSE), while **MAE** is the mean absolute error and is less sensitive to outliers. Training and five-fold CV error are listed together so they can be read side by side — the training figures are optimistic because the model was scored on the same rows it was fitted on, and the cross-validation note below explains how the CV figure is built and what the gap between the two means.

```
# 6. Residual / influence diagnostic: Cook's distance
cooks_d <- cooks.distance(ols_fit)
n_train <- nrow(train_data)
threshold <- 4 / n_train

plot(cooks_d,
     type = "h", lwd = 1.5, col = "steelblue",
     ylab = "Cook's distance", xlab = "Observation index",
     main = "Cook's Distance - OLS Baseline")
abline(h = threshold, lty = 2, col = "firebrick", lwd = 1.5)
text_idx <- which(cooks_d > threshold)
if (length(text_idx)) {
  text(text_idx, cooks_d[text_idx],
       labels = text_idx, pos = 3, cex = 0.8, col = "firebrick")
}
```

```
cat(sprintf("Threshold (4/n):    %.4f\n", threshold))
```

```
## Threshold (4/n):    0.0519
```

```
cat(sprintf("Influential points: %d of %d\n", length(text_idx), n_train))
```

```
## Influential points: 7 of 77
```

**Model summary.** The OLS model is fitted on the 77 standardized training observations with all 8 predictors. Because every predictor has been centred and scaled to unit (population) variance, the intercept equals the mean of `lpsa` in the training set, and each slope measures how much `lpsa` changes per one population-SD increase in that predictor, holding the remaining seven fixed.

**Coefficient interpretation.** Two predictors dominate.

- `lcavol` carries the largest standardized coefficient ( $\hat{\beta} \approx 0.72$ ,  $p < 0.001$ ), confirming that log cancer volume is the single strongest linear predictor of log PSA.

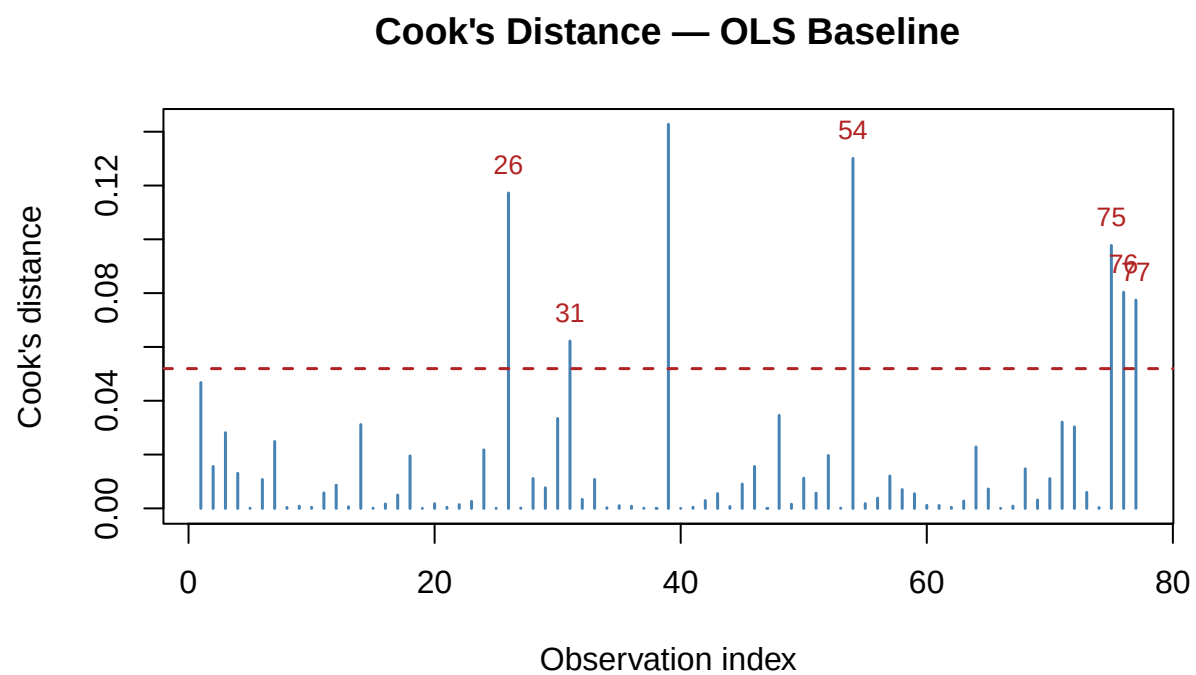


Figure 1: Cook's distance for the OLS fit. Observations exceeding the  $4/n$  threshold (dashed line) are flagged as influential.

- `svi` is the second-largest contributor ( $\hat{\beta} \approx 0.27$ ,  $p \approx 0.021$ ), indicating that seminal vesicle invasion is associated with meaningfully higher PSA even after adjusting for tumour volume.
- `lweight` is marginally significant ( $p \approx 0.041$ ); the remaining 5 predictors (`age`, `lbph`, `lcp`, `gleason`, `pgg45`) all have  $|t| < 1.4$  and  $p > 0.16$ .

**Collinearity concern.** The near-zero and sign-reversed coefficient for `lcp` ( $\hat{\beta} \approx -0.04$ ) is noteworthy: biologically, greater capsular penetration should correlate positively with PSA, and indeed the marginal correlation is positive. The sign flip in OLS is a classic symptom of multicollinearity — `lcavol` and `lcp` are strongly correlated ( $r \approx 0.68$  on the training set), so OLS distributes their shared variance unevenly. Regularization in 2B–2C is expected to stabilize these shared-variance coefficients.

**Cross-validation error.** The five-fold CV RMSE (0.794) is computed using the same `foldid` assignment that will be passed to `cv.glmnet` for ridge and lasso, ensuring a like-for-like comparison. Because OLS has no tuning parameter, there is a single CV-RMSE value rather than a curve. It exceeds the training RMSE (0.681) by only about 0.113 on the log-PSA scale. This modest optimism gap reflects mild in-sample overfitting: it is small enough to suggest the 8-predictor model already generalizes reasonably well on the 77-row training sample, and 2B–2C will test whether shrinkage narrows the gap further.

**Influence diagnostic.** The Cook’s distance plot flags observations whose removal would substantially shift the fitted coefficients. The conventional threshold  $4/n$  is shown as a dashed line. Any point above this line has outsized leverage, large residual, or both. In a sample of only 77 training rows, even one or two influential points can destabilize an unpenalized fit — particularly for the collinear predictors `lcavol`/`lcp` whose coefficient estimates are already fragile. These influential observations are retained in the analysis (not removed) because they are clinically valid patients; their effect is instead mitigated by the regularization introduced in 2B–2C.

## 2.2 2B. Ridge

```
# Use fit_cv_glmnet(..., alpha = 0, foldid = foldid).
# Report lambda.min, lambda.1se, paths, coefficients, and CV evidence.
```

## 2.3 2C. Lasso

```
# Use fit_cv_glmnet(..., alpha = 1, foldid = foldid).
# Report lambda.min, lambda.1se, paths, and nonzero predictors.
```

## 2.4 2D. Training-based comparison and locked core model

```
# Build a comparison from training/CV quantities only.
```

Core model nominated before holdout evaluation: [method and reason]

## 2.5 2E. Perturbation or stability check

Prediction before running the check: [prediction]

```
# Implement one allowed training-only stability check.
```

Observed result and explanation: [result]

### 3 Problem 3. Mathematical mechanisms

#### 3.1 3A. Ridge and conditioning

Write the ridge derivation. Define dimensions and explain uniqueness.

```
# Use x_train and the fitted ridge lambda.min.  
# safe_condition_numbers(x_train, lambda = ...)
```

#### 3.2 3B. Lasso optimality and soft thresholding

Derive the zero/nonzero optimality conditions and the orthonormal-design formula.

#### 3.3 3C. Connect algebra to evidence

Connect one specific numerical result to Problem 3A or 3B.

### 4 Problem 4. Elastic net and a neural-feature bridge

#### 4.1 4A. Research question and source map

Research direction: [weight decay / sparse weights / pruning / dropout]

| Claim   | Exact primary source<br>and locator | What it establishes | What it does not<br>establish |
|---------|-------------------------------------|---------------------|-------------------------------|
| [Claim] | [Citation, section/page/equation]   | [Support]           | [Boundary]                    |

#### 4.2 4B. Elastic net on original predictors

```
alpha_grid <- c(0.10, 0.25, 0.50, 0.75, 0.90)  
  
# Fit one cv.glmnet object per alpha with the same foldid.  
# Select alpha and lambda using training CV only.
```

#### 4.3 4C. Fixed ReLU features

```
relu <- function(z) pmax(z, 0)  
M <- 200L  
  
# TODO:  
# 1. Generate A and bias once from seeds$features using the stated distributions.
```

```
# 2. Apply the same A and bias to x_train and x_test.
# 3. Fit a scaler to H_train only and apply it to H_test.
# 4. Fit ridge, lasso, and elastic net on H_train with the shared foldid.
```

State which weights are fixed, which are learned, and how active features are counted.

#### 4.4 4D. Locked declaration and final holdout evaluation

**Locked before viewing y\_test:** [preprocessing, candidate specifications, selected tuning values, feature seed, nominated deployment model, metrics]

```
# This must be the first analysis chunk that uses y_test for evaluation.
# Generate all fixed candidate predictions and one RMSE/MAE table here.
```

State whether the holdout evidence supports the predeclared choice. Do not retune.

#### 4.5 4E. Deep-learning connection and limitations

Distinguish penalty, weight decay, output-weight sparsity, pruning, dropout, and a trained deep network. Cite the sources from 4A and discuss at least two limitations.

### 5 Problem 5. Report quality and reproducibility

#### 5.1 5A. Reproduction record

1. Open a clean R session.
2. [Exact commands and folder layout.]
3. Render GroupXX\_ProjectQuiz1.Rmd.

#### 5.2 5B. Recommendation and limitations

Give the final recommendation, distinguish prediction from interpretation, and state limitations.

#### 5.3 5C. AI verification record

| AI-assisted item | How it was checked | Evidence             | Modification or rejection |
|------------------|--------------------|----------------------|---------------------------|
| [Item]           | [Method]           | [Test/source/output] | [Decision]                |

### Preflight and session information

```
# Add every object required to reproduce the final table and figures.
required_objects <- c(
  "train_id", "test_id", "x_train", "x_test", "y_train", "y_test", "foldid"
)
missing_objects <- setdiff(required_objects, ls())
```

```

if (length(missing_objects)) {
  stop(
    "Missing required objects: ",
    paste(missing_objects, collapse = ", ")
  )
}
stopifnot(length(intersect(train_id, test_id)) == 0L)

```

```

sessionInfo()

```

```

## R version 4.6.0 (2026-04-24 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26200)
##
## Matrix products: default
##   LAPACK version 3.12.1
##
## locale:
## [1] LC_COLLATE=Vietnamese_Vietnam.utf8  LC_CTYPE=Vietnamese_Vietnam.utf8
## [3] LC_MONETARY=Vietnamese_Vietnam.utf8 LC_NUMERIC=C
## [5] LC_TIME=Vietnamese_Vietnam.utf8
##
## time zone: Asia/Saigon
## tzcode source: internal
##
## attached base packages:
## [1] stats      graphics  grDevices  utils      datasets  methods    base
##
## other attached packages:
##  [1] knitr_1.51      broom_1.0.13   glmnet_5.0     Matrix_1.7-5
##  [5] lubridate_1.9.5 forcats_1.0.1  stringr_1.6.0  dplyr_1.2.1
##  [9] purrr_1.2.2     readr_2.2.0    tidyr_1.3.2    tibble_3.3.1
## [13] ggplot2_4.0.3   tidyverse_2.0.0
##
## loaded via a namespace (and not attached):
##  [1] generics_0.1.4   shape_1.4.6.1   stringi_1.8.7   lattice_0.22-9
##  [5] hms_1.1.4        digest_0.6.39   magrittr_2.0.5   timechange_0.4.0
##  [9] evaluate_1.0.5   grid_4.6.0      RColorBrewer_1.1-3 iterators_1.0.14
## [13] fastmap_1.2.0    foreach_1.5.2   backports_1.5.1  survival_3.8-6
## [17] scales_1.4.0     codetools_0.2-20 cli_3.6.6        rlang_1.2.0
## [21] splines_4.6.0    withr_3.0.2     yaml_2.3.12      otel_0.2.0
## [25] tools_4.6.0      tzdb_0.5.0      vctrs_0.7.3      R6_2.6.1
## [29] lifecycle_1.0.5  pkgconfig_2.0.3 pillar_1.11.1    gtable_0.3.6
## [33] glue_1.8.1       Rcpp_1.1.2      xfun_0.57        tidyselect_1.2.1
## [37] rstudioapi_0.18.0 farver_2.1.2     htmltools_0.5.9  rmarkdown_2.31
## [41] compiler_4.6.0   S7_0.2.2

```

Stamey, Thomas A., John N. Kabalin, John E. McNeal, et al. 1989. "Prostate Specific Antigen



in the Diagnosis and Treatment of Adenocarcinoma of the Prostate. II. Radical Prostatectomy Treated Patients.” *The Journal of Urology* 141 (5): 1076–83. [https://doi.org/10.1016/S0022-5347\(17\)41175-X](https://doi.org/10.1016/S0022-5347(17)41175-X).